

Guía del Proyecto Final Integrador

Demostración de Competencias Profesionales en Desarrollo de Software

Estimados estudiantes, ha llegado el momento de consolidar y demostrar todo el conocimiento adquirido a lo largo de los últimos tres módulos. Este proyecto final es su carta de presentación como desarrolladores y analistas; en él pondrán en práctica desde la planeación y el diseño de la arquitectura hasta el despliegue en un entorno real.

Lineamientos Generales

- **Temática del Proyecto:** ¡Libre! Tienen la oportunidad de proponer una solución tecnológica innovadora para el problema que deseen.
- **Stack Tecnológico:** Es libre. Sin embargo, se recomienda firmemente utilizar el ecosistema trabajado en clase: **Python (Django o FastAPI)**, **Base de Datos (MySQL o SQLite)** y **Frontend (HTML, CSS y JavaScript)**.

Restricción Importante: Para fomentar la innovación y salir de las soluciones comunes, **NO se aceptarán proyectos de: Puntos de Venta (POS), Sistemas de Inventario o Comercio Electrónico (E-commerce).**



Requisitos del Proyecto y Ejemplos de Referencia

A continuación, se detalla lo que deben entregar por cada módulo, acompañado de un ejemplo basado en un **E-commerce de ropa** para que les sirva de guía (recuerden que no pueden elegir este tema).



MÓDULO 1: Análisis y Diseño de Sistemas

En esta etapa deben demostrar que entienden el problema antes de escribir la primera línea de código.

1. Informe Técnico Completo

Debe incluir:

- **Propósito:** El "por qué" y "para qué" existe el sistema.
- **Alcance:** Qué hace y, muy importante, qué **no** hace el sistema en esta primera versión.
- **Flujo de Usuarios y Caminos (Paths):**
 - **Happy Path (Camino feliz):** El flujo ideal donde el usuario logra su objetivo sin errores.
 - **Unhappy Path (Camino triste):** Flujo donde ocurre un error o el usuario cancela la acción.
 - **Alternative Path (Camino alternativo):** Una ruta diferente para lograr el mismo objetivo.



Ejemplo para un E-commerce:

- **Propósito:** Permitir a usuarios locales adquirir prendas de vestir de forma 100% online, reduciendo el tiempo de compra a menos de 5 minutos.
- **Alcance:** El sistema permitirá registrarse, buscar ropa por categorías, añadir al carrito y pagar con tarjeta simulada. *Fuera de alcance:* No gestionará envíos por paquetería externa ni devoluciones automáticas en esta fase.
- **Happy Path (Compra Exitosa):** El usuario inicia sesión ➡ busca una camiseta ➡ la añade al carrito ➡ ingresa su tarjeta ➡ el pago se procesa con éxito ➡ recibe confirmación.
- **Unhappy Path (Tarjeta Rechazada):** El usuario intenta pagar ➡ la pasarela rechaza la tarjeta por fondos insuficientes ➡ el sistema muestra un mensaje de error y mantiene el carrito intacto sin generar la orden.
- **Alternative Path (Compra con Cupón):** El usuario añade la camiseta al carrito ➡ antes de pagar, aplica un cupón de descuento del 10% ➡ el sistema recalcula el total ➡ el usuario paga el nuevo monto con éxito.

2. Diagrama de Flujo

Representación lógica de los procesos del sistema mediante simbología estándar (procesos, decisiones, inicio/fin).

Ejemplo para un E-commerce:

Un diagrama que inicia cuando el cliente entra al sitio, pasa por una decisión (*¿Está autenticado?*), deriva en el formulario de login o directo al checkout, y finaliza con la generación de la factura.

3. Diagrama de Interfaz y Prototipos de Alta Fidelidad

- **Diagrama de Interfaz (Wireframes / Mockups):** El esqueleto o estructura de las pantallas en gris/blanco y negro para entender la distribución.
- **Prototipo de Alta Fidelidad:** El diseño visual final con colores, tipografías y enlaces interactivos que simulen el funcionamiento de la app (herramientas recomendadas: Figma o Penpot).

Ejemplo para un E-commerce:

Diseñar la pantalla de la "Tienda principal". El wireframe muestra dónde irá el logo, la barra de búsqueda y la cuadrícula de productos. El prototipo de alta fidelidad muestra esa misma pantalla con los colores reales de la marca, las fotos de la ropa, botones llamativos de "Añadir al carrito" y transiciones clicables que llevan al usuario a la vista del producto.

MÓDULO 2: Base de Datos

La estructura de datos debe ser sólida, óptima y estar perfectamente justificada.

1. Diagrama Entidad-Relación Físico (DER)

El esquema técnico final que se creará en el motor de bases de datos, detallando tablas, llaves primarias (PK), llaves foráneas (FK), tipos de datos y cardinalidad.

2. Documentación de Restricciones y Justificación

Deben listar las reglas de negocio aplicadas a nivel de base de datos (`NOT NULL` , `UNIQUE` , `CHECK` , `FOREIGN KEY` con borrado en cascada o restringido) y explicar el porqué de cada decisión.



Ejemplo para un E-commerce:

- **Tabla Usuarios** : El campo `email` se define como `VARCHAR(150) UNIQUE NOT NULL` .
Justificación: El correo es obligatorio para enviar las facturas y debe ser único para evitar que dos personas compartan la misma cuenta.
- **Tabla Detalles_Orden** : El campo `cantidad` tiene una restricción `CHECK (cantidad > 0)` .
Justificación: No se pueden realizar ventas de 0 o cantidades negativas de productos; esto rompería la lógica del inventario y las finanzas.
- **Relación Productos** → **Detalles_Orden** : Llave foránea con restricción `ON DELETE RESTRICT` .
Justificación: Si un producto ya fue vendido en el pasado, no se puede eliminar de la base de datos de forma permanente, ya que dejaría las facturas viejas sin información del artículo vendido.



MÓDULO 3: Desarrollo de Sistemas Informáticos

La fase donde el diseño y la lógica cobran vida en el código.

1. Informe de Especificaciones Técnicas

Un documento conciso que detalle:

- **Stack de Desarrollo:** Las herramientas exactas, lenguajes, frameworks y librerías utilizadas con sus respectivas versiones (por ejemplo: Python 3.11, Django 5.0, SQLite3, Bootstrap 5).
- **Modelos de Datos:** La traducción del DER físico a código (por ejemplo, las clases de modelos en Django o SQLAlchemy).

Ejemplo para un E-commerce:

El informe detalla que se usó Django debido a su robusto sistema de autenticación integrado. Muestra el código del modelo `Producto` en Python:

```
class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField()
```

2. Subida al Servidor del Proyecto (Despliegue)

El sistema no puede quedarse corriendo únicamente en sus computadoras (`localhost`). Debe ser desplegado en un servidor web real y accesible mediante un enlace público para su evaluación.

Ejemplo para un E-commerce:

Subir la aplicación a plataformas como Render, Railway, PythonAnywhere o un VPS, conectando la aplicación a una base de datos en la nube (como Supabase o Clever Cloud) para que cualquiera pueda entrar a la tienda desde su teléfono o navegador mediante una URL activa.

3. Presentación Final

Una defensa profesional del proyecto frente a la clase. Deberán exponer la problemática, el análisis, la arquitectura de datos y realizar una **demostración en vivo (Live Demo)** del sistema funcionando directamente desde el enlace del servidor.



Criterios de Evaluación

1. **Calidad del Análisis (Módulo 1):** Correspondencia exacta entre los diagramas, prototipos y el software final entregado.
2. **Integridad de la Base de Datos (Módulo 2):** Estructura correcta, normalización y aplicación estricta de las restricciones de integridad.

3. **Funcionalidad y Despliegue (Módulo 3):** Código limpio, cumplimiento del alcance prometido, ausencia de errores críticos en producción y fluidez en la presentación.



ATENCIÓN: Las exposiciones de los proyectos comenzarán el día 20 de junio.